

MACHINE LEARNING LABORATORY

Lab Handout — Session 3 Hours

Lab Objectives

- Understand hierarchical (agglomerative) clustering and interpret dendrograms.
- Understand the concept of Ensemble Learning and why combining multiple models improves performance.
- Implement and compare Bagging (Bootstrap Aggregating) with a single base learner.
- Implement AdaBoost and understand how boosting sequentially corrects errors.
- Practice with real scikit-learn datasets and evaluate models using standard metrics.

Session Schedule (Total: 3 Hours)

Time	Activity	Duration
00:00 – 00:10	Introduction: Recap of clustering & ensemble concepts	10 min
00:10 – 01:00	Part 1 — Agglomerative Clustering (theory + hands-on code)	50 min
01:00 – 01:50	Part 2 — Ensemble Learning: Bagging (theory + hands-on code)	50 min
01:50 – 02:00	Short Break	10 min
02:00 – 02:50	Part 3 — Boosting & AdaBoost (theory + hands-on code)	50 min
02:50 – 03:00	Lab Assignment briefing, Viva questions & wrap-up	10 min

Part 1 — Agglomerative Clustering

1.1 Theory

Agglomerative Clustering is a bottom-up hierarchical clustering technique. It starts by treating every data point as its own cluster, and repeatedly merges the two closest clusters until only one cluster (or a specified number of clusters) remains. The sequence of merges is recorded and visualized using a dendrogram — a tree diagram where the height of each merge represents the distance between the merged clusters.

Linkage Criteria (how distance between clusters is measured):

- Single Linkage — minimum distance between points of two clusters.
- Complete Linkage — maximum distance between points of two clusters.
- Average Linkage — average distance between all pairs of points.
- Ward's Linkage — merges the pair of clusters that leads to the minimum increase in total within-cluster variance (most commonly used, produces compact clusters).

The number of final clusters can be chosen by “cutting” the dendrogram horizontally at a chosen height — the number of vertical lines the cut crosses equals the number of clusters.

1.2 Dataset

We use the classic Iris dataset (built into scikit-learn): 150 samples of iris flowers, 4 numeric features (sepal length, sepal width, petal length, petal width), and 3 true species classes (Setosa, Versicolor, Virginica) used only for evaluation, not for training, since clustering is unsupervised.

1.3 Code Example

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import silhouette_score, adjusted_rand_score
from scipy.cluster.hierarchy import dendrogram, linkage

# 1. Load dataset
iris = load_iris()
X = iris.data
y = iris.target          # true species (used only for evaluation)

# 2. Standardize features (important: distance-based algorithm)
X_scaled = StandardScaler().fit_transform(X)

# 3. Build linkage matrix and plot dendrogram (Ward linkage)
Z = linkage(X_scaled, method='ward')

plt.figure(figsize=(10, 5))
dendrogram(Z, truncate_mode='lastp', p=15)
plt.title('Dendrogram (Ward Linkage) - Iris Dataset')
plt.xlabel('Sample index / (cluster size)')
plt.ylabel('Distance')
plt.show()

# 4. Fit Agglomerative Clustering with 3 clusters
model = AgglomerativeClustering(n_clusters=3, linkage='ward')
```

```

labels = model.fit_predict(X_scaled)

# 5. Evaluate
sil = silhouette_score(X_scaled, labels)
ari = adjusted_rand_score(y, labels)
print('Silhouette Score:', round(sil, 3))
print('Adjusted Rand Index vs true species labels:', round(ari, 3))
print('Cluster label counts:', np.bincount(labels))

```

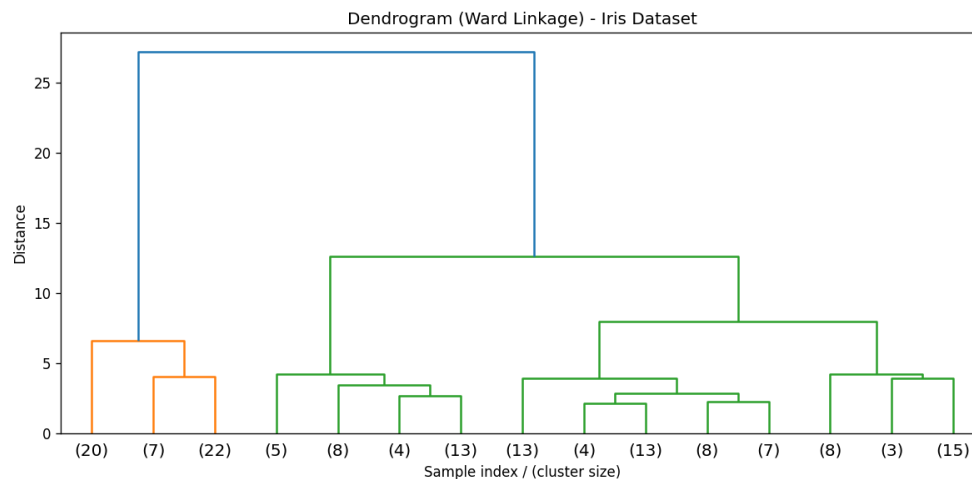


Figure 1.1 — Dendrogram of Iris data (truncated to last 15 merges), Ward linkage.

1.4 Sample Output & Interpretation

Metric	Value
Silhouette Score	≈ 0.447
Adjusted Rand Index (vs species)	≈ 0.615
Cluster sizes found	[71, 49, 30]

A silhouette score close to 1 indicates well-separated, compact clusters; values around 0.4–0.5 indicate reasonable but overlapping structure — expected here since Versicolor and Virginica overlap in feature space. The Adjusted Rand Index (ARI) compares the discovered clusters to the true species labels purely for evaluation purposes; an ARI of 0.615 shows substantial agreement even though the algorithm never saw the labels.

1.5 Lab Exercise

1. Repeat the experiment using 'single' and 'complete' linkage. Compare the dendrograms and silhouette scores with Ward linkage.
2. Try AgglomerativeClustering with `n_clusters = 2` and `n_clusters = 4`. Which value best matches the natural structure of the data?
3. Apply Agglomerative Clustering to the Wine dataset (`sklearn.datasets.load_wine`) and report your observations.

Part 2 — Ensemble Learning: Bagging

2.1 Theory

Ensemble Learning combines predictions from multiple base models (“weak learners”) to produce a stronger, more robust model. The key idea is that a group of moderately accurate, diverse models can outperform any single model by reducing variance (overfitting) and/or bias.

Bagging (Bootstrap Aggregating):

- Multiple bootstrap samples (random samples drawn with replacement) are created from the training data.
- An independent base model (e.g., a Decision Tree) is trained on each bootstrap sample, in parallel.
- Final prediction = majority vote (classification) or average (regression) across all base models.
- Primarily reduces variance — very effective with high-variance, low-bias models like unpruned Decision Trees.
- Random Forest is a well-known extension of Bagging that also randomizes the feature subset considered at each split.

Note:

Bagging trains base learners independently and in parallel on different random subsets of data. Boosting (Part 3) trains them sequentially, where each new learner focuses on the mistakes of the previous ones.

2.2 Dataset

We use the Breast Cancer Wisconsin (Diagnostic) dataset (built into scikit-learn): 569 samples, 30 numeric features computed from digitized images of breast tissue, and a binary target (malignant / benign).

2.3 Code Example

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import BaggingClassifier, RandomForestClassifier
from sklearn.metrics import accuracy_score

# 1. Load dataset
data = load_breast_cancer()
X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

# 2. Baseline: a single Decision Tree
tree = DecisionTreeClassifier(random_state=42)
tree.fit(X_train, y_train)
tree_pred = tree.predict(X_test)
print('Single Decision Tree Accuracy:', round(accuracy_score(y_test, tree_pred), 4))

# 3. Bagging Classifier (50 bootstrapped trees)
bagging = BaggingClassifier(
    estimator=DecisionTreeClassifier(random_state=42),
    n_estimators=50,
    max_samples=0.8,
    bootstrap=True,
    random_state=42
)
```

```

bagging.fit(X_train, y_train)
bag_pred = bagging.predict(X_test)
print('Bagging Classifier Accuracy:', round(accuracy_score(y_test, bag_pred), 4))

# 4. Random Forest (Bagging + random feature subsets)
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
rf_pred = rf.predict(X_test)
print('Random Forest Accuracy:', round(accuracy_score(y_test, rf_pred), 4))

```

2.4 Sample Output

Model	Test Accuracy
Single Decision Tree	0.9231
Bagging Classifier (50 trees)	0.9650
Random Forest (100 trees)	0.9580

Notice both ensemble methods outperform the single Decision Tree. A single unpruned tree tends to overfit the training data (high variance); averaging many trees trained on different bootstrap samples smooths out this variance and gives more stable, generalizable predictions.

2.5 Lab Exercise

4. Vary `n_estimators` (e.g., 5, 10, 50, 100, 200) in `BaggingClassifier` and plot test accuracy vs number of estimators.
5. Replace the base estimator with a `k-Nearest-Neighbors` classifier. Does Bagging still help? Why or why not?
6. Compare `BaggingClassifier` and `RandomForestClassifier` training time and accuracy on the same split.

Part 3 — Boosting & AdaBoost

3.1 Theory — Boosting

Boosting builds an ensemble sequentially: each new base learner is trained to focus on the mistakes made by the previous learners, rather than being trained independently. The learners are typically weak (only slightly better than random guessing), but combining them sequentially and adaptively produces a strong overall model. Boosting primarily reduces bias, and can also reduce variance.

Bagging vs Boosting:

Aspect	Bagging	Boosting
Training of base learners	Parallel, independent	Sequential, dependent
Data sampling	Bootstrap samples (equal weight)	Full data, re-weighted per round
Goal	Reduce variance	Reduce bias (and variance)
Combination	Simple vote / average	Weighted vote (better learners weigh more)
Example algorithm	Random Forest	AdaBoost, Gradient Boosting, XGBoost

3.2 AdaBoost (Adaptive Boosting) — Algorithm

7. Initialize equal weights for all N training samples: $w_i = 1/N$.
8. For each round $t = 1 \dots T$: train a weak learner (commonly a decision stump — a tree of depth 1) on the weighted training data.
9. Compute the weighted error rate of that learner: the fraction of weighted samples it misclassified.
10. Compute the learner's importance/weight (α) — lower error gives a higher α (more say in the final vote).
11. Increase the weights of the misclassified samples and decrease weights of correctly classified samples, so the next learner focuses more on the hard cases.
12. Repeat for T rounds. Final prediction = weighted majority vote of all weak learners, using their α values.

3.3 Dataset

We continue with the Breast Cancer Wisconsin dataset for direct comparability with Part 2, using the same train/test split.

3.4 Code Example

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score

data = load_breast_cancer()
X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)
```

```
# 1. Weak learner alone: a Decision Stump (max_depth = 1)
stump = DecisionTreeClassifier(max_depth=1, random_state=42)
stump.fit(X_train, y_train)
stump_pred = stump.predict(X_test)
print('Single Decision Stump Accuracy:', round(accuracy_score(y_test, stump_pred), 4))

# 2. AdaBoost using 100 decision stumps
ada = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1, random_state=42),
    n_estimators=100,
    learning_rate=1.0,
    random_state=42
)
ada.fit(X_train, y_train)
ada_pred = ada.predict(X_test)
print('AdaBoost Accuracy:', round(accuracy_score(y_test, ada_pred), 4))

# 3. Accuracy as more weak learners are added (staged predictions)
staged_acc = list(ada.staged_score(X_test, y_test))

plt.figure(figsize=(8, 5))
plt.plot(range(1, len(staged_acc) + 1), staged_acc, color='darkorange')
plt.xlabel('Number of Weak Learners (Estimators)')
plt.ylabel('Test Accuracy')
plt.title('AdaBoost: Accuracy vs Number of Estimators')
plt.grid(alpha=0.3)
plt.show()

print('First 10 estimator weights (alpha):', np.round(ada.estimator_weights_[:10], 3))
```

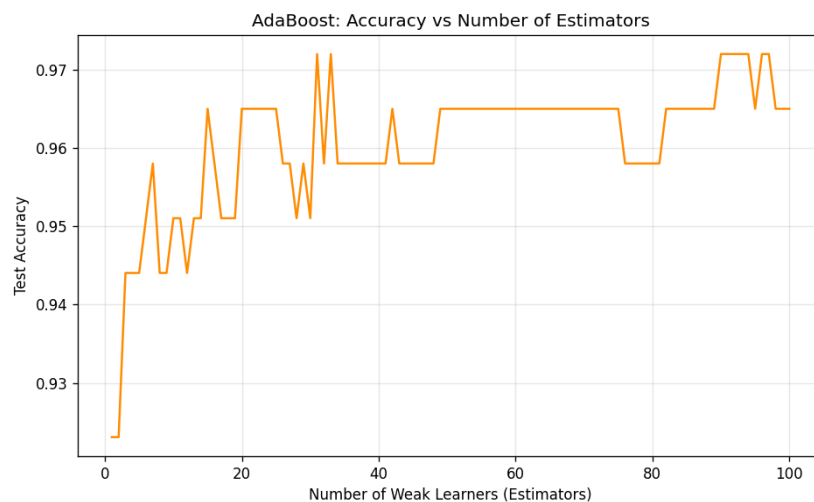


Figure 3.1 — Test accuracy improves and stabilizes as more weak learners are added.

3.5 Sample Output & Interpretation

Model	Test Accuracy
Single Decision Stump (depth = 1)	0.9231
AdaBoost (100 stumps)	0.9650

A single decision stump is a very weak model, yet 100 of them combined through AdaBoost match the performance of the full Bagging ensemble from Part 2. Notice the estimator weights (alpha values) — early rounds typically receive

higher alpha because those stumps make fewer weighted errors before the sample weights get heavily redistributed toward hard cases.

3.6 Lab Exercise

13. Plot the `estimator_errors_` and `estimator_weights_` arrays across all 100 rounds. What pattern do you notice?
14. Re-run AdaBoost with `learning_rate = 0.5` and `learning_rate = 1.5`. How does it affect convergence and final accuracy?
15. Replace the base estimator with `max_depth = 3` trees instead of stumps. Compare accuracy and discuss overfitting risk.
16. Try `GradientBoostingClassifier` from `sklearn.ensemble` on the same dataset and compare with AdaBoost.

Take-Home Lab Assignment

Submit a short Jupyter Notebook (with outputs) covering the following, using the Wine dataset (`sklearn.datasets.load_wine`):

17. Apply Agglomerative Clustering with all four linkage methods and report the best one (using silhouette score).
18. Train a BaggingClassifier and a RandomForestClassifier; compare accuracy, precision, recall, and F1-score using `classification_report`.
19. Train an AdaBoostClassifier and tune `n_estimators` and `learning_rate` using GridSearchCV. Report the best combination.
20. Write a half-page summary comparing all methods used in this lab, including when you would prefer clustering vs. classification, and Bagging vs. Boosting.

Viva Voce / Quick Review Questions

- Why must features be standardized before Agglomerative Clustering, but this is less critical for tree-based ensembles?
- What is the difference between a dendrogram and a cluster assignment?
- Why does Bagging mainly reduce variance while Boosting mainly reduces bias?
- What is a 'weak learner' in AdaBoost, and why is a decision stump commonly used?
- How are sample weights updated between rounds in AdaBoost?
- Why can Boosting be more prone to overfitting on noisy data than Bagging?

References

scikit-learn documentation: Clustering — <https://scikit-learn.org/stable/modules/clustering.html>

scikit-learn documentation: Ensemble methods — <https://scikit-learn.org/stable/modules/ensemble.html>

Freund, Y. & Schapire, R. (1997). A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting.